23 THREADS (HILOS).

La forma en la que Java es capaz de implementar varios procesos simultáneamente (multiprocesamiento o multithreading) es a través de hilos.

FORMAS PRINCIPALES DE INSTANCIAR UN HILO:

1. Creando una clase que herede de Thread y sobrescribiendo el método public void run() {} ... no puede lanzar excepciones porque el original no lanza ninguna. La herencia hace que dicha clase sea también un Thread (recordar que ya no podremos usar más la herencia directa).

```
MiHilo h1 = new MiHilo();
```

2. Creando una clase que implemente la interfaz Runnable (obliga a implementar el método public void run()) y pasando una instancia de la misma al constructor de Thread (sobrecargado). Hay que destacar que dicha clase NO ES UN THREAD, sino un RUNNABLE, por lo que no tendremos los métodos heredados de Thread. A cambio, seguiremos teniendo disponible el uso de la herencia directa.

```
Thread t1 = new Thread(new MiRunnable());

// LO MISMO PERO CON DECLARACIÓN EN LÍNEA DE LA INTERFAZ Y SU CONTENIDO:
Thread t3 = new Thread(
                    new Runnable() {
                        public void run() {
                            // código de ejecución del hilo //
                        }
                    }
                );
```

También puede utilizarse el constructor que pida el nombre con el que designar al hilo o bien utilizarse más tarde el método setName(“”).

```
Thread(Runnable r, String nombre)
Thread(String nombre)
```

Todo hilo es gestionado por un elemento denominado **SCHEDULER** o planificador, que se encargará de activar o desactivar estos (ejecutar a lo largo del tiempo el contenido de su método run()) en lo que se conoce como CICLO DE VIDA DE UN HILO.

El funcionamiento, el plan de ejecución de dicho planificador ES TOTALMENTE INDEPENDIENTE DE LA VOLUNTAD DEL PROGRAMADOR y aunque existen algunos métodos con los que tratar de “sugerirle” cómo debe actuar, será éste el que finalmente decida cuál es el hilo que debe estarse ejecutando ... en resumen, “EL PLANIFICADOR ES EL QUE MANDA”.

Así, podríamos decir que cuando se trata de hilos, NO HAY NADA 100% SEGURO RESPECTO A LA EJECUCIÓN DE LOS MISMOS. Con hilos suele ser habitual que ejecutar dos veces el mismo programa, en la misma máquina virtual, genere distintos resultados. Aparte, cada versión de dicha máquina virtual, puede tener su propia versión del planificador con lo que los resultados entre ellos variará casi con toda seguridad.

EL CICLO DE VIDA DE UN HILO:

ESTADOS NORMALES:

1.	NEW	- INSTANCIADO:	Instanciar un objeto Thread. El futuro hilo aún no está vivo ni tiene pila de ejecución. El método isAlive() indicaría false.
2.	RUNNABLE	- DISPONIBLE:	Llamar a su método start(). Hace que el planificador gestione y entregue al objeto Thread una pila de ejecución (Execution Stack) y lo marque como hilo elegible. El hilo está vivo y a punto para poder ejecutar el contenido de su run() al ser elegido. El método isAlive indicaría true.
3.	RUNNING	- EJECUTÁNDOSE:	Ejecuta el método run() del hilo seleccionado (variables a su pila de ejecución).

ESTADOS DE NO EJECUCIÓN (NON RUNNING STATES):

4.	WAITING	- ESPERANDO:	Hilo en espera de que otro le despierte o pase un cierto tiempo. Suelta la llave.
5.	SLEEPING	- DURMIENDO:	Hilo dormido hasta que pase como mínimo el tiempo indicado. No suelta la llave.
6.	BLOCKED	- BLOQUEADO:	Hilo en espera hasta que se produzca situación de desbloqueo. Acceso a dispositivo.

ESTADO FINAL:

7.	DEAD	- MUERTO	El método run() del hilo ha sido completado. No se puede volver a activar. El método isAlive indicaría false.

Como mención especial, tener en cuenta que un hilo pasa siempre del estado RUNNING al estado RUNNABLE o en su defecto a uno de los NON RUNNING STATES o al DEAD. De igual forma, pasará de un NON RUNNING STATE al estado RUNNABLE (siempre y cuando se produzca el adecuado evento que provoque el cambio).

Lo que nunca va a poder hacer es pasar al estado de ejecución (RUNNING) directamente desde un estado que no sea el de disponible (RUNNABLE) y sin la actuación del planificador.

En definitiva, el paso al estado de ejecución (RUNNING) sólo puede hacerse a través del planificador, debiendo estar el hilo previamente en estado RUNNABLE (disponible o elegible).

HERRAMIENTAS PARA SINCRONIZACIÓN SOBRE UN RECURSO COMPARTIDO (BLOQUE O MÉTODO synchronized):

DEBEMOS USARLOS SOBRE EL RECURSO COMPARTIDO, o de lo contrario obtendremos una IllegalThreadStateException:

Si bloque de código sincronizado a través de un objeto compartido	--->	**objeto.**wait();
Si método sincronizado (se usa la propia instancia)	--->	wait(); ... es this implícito (no hace falta ponerlo).

wait() Poner el hilo en espera, WAITING, hasta que éste reciba una notificación por parte del planificador (como solicitud de otro hilo sobre el recurso compartido por ambos) para pasar a RUNNABLE.
Puede lanzar InterruptedException.

wait(tiempo ms)
Igual que el anterior, pero pasará al estado RUNNABLE bien por notificación bien porque haya pasado como mínimo el tiempo indicado.
Puede lanzar InterruptedException.

notify() La forma en la que un hilo notifica al planificador que ponga en RUNNABLE otro hilo de los que comparten con él, un determinado recurso.

notifyAll()
La forma en la que un hilo notifica al planificador que ponga en RUNNABLE todos aquellos hilos que comparten con el un determinado recurso.

ESTOS MÉTODOS NO SON PROPIOS DE THREAD, SINO DE OBJECT.

HAN DE USARSE DENTRO DE UN BLOQUE O MÉTODO SINCRONIZADO (palabra reservada synchronized u obtendremos un error de compilación)

El utilizar la sincronización con un recurso compartido evita los problemas propios del multiacceso, como podría ser la inconsistencia de datos. Para ello, Java provee los denominados cerrojos o llaves, de tal forma que sólo existe una llave por cada elemento compartido y sólo un hilo puede estar en posesión de dicha llave. Aquel hilo que disponga de la llave, podrá ejecutar el código declarado con la palabra synchronized. El resto de hilos, a pesar de estar en un determinado momento en ejecución, no podrán ejecutar el código del bloque o método compartido.

Hay que resaltar el hecho de que una llamada a sleep() provocará que el hilo que "se pone a dormir", se lleve consigo la llave sobre el recurso compartido, en el caso de tenerla. Sin embargo, una llamada a wait(), hará que el hilo "se ponga en espera" pero previamente entregando la llave al planificador para que éste se la pueda dar al siguiente hilo que entre en ejecución, pudiendo así, hacer uso del código sincronizado (sólo si este trabaja también con dicho recurso compartido).

Tanto wait como sleep (además del método de encolamiento join) pueden provocar una excepción de tipo InterruptedException.

Las denominadas aplicaciones productor – consumidor hacen uso de la sincronización para que el hilo productor (el que genera datos) actué conjuntamente con el hilo consumidor (el que usa dichos datos) sin caer en los problemas de consistencia propios de un acceso concurrente o simultaneo. Así, se define el recurso compartido, que será un objeto de una determinada clase (como por ejemplo un File) que le será pasado a ambos hilos a la hora de construirlos. La invocación a los métodos de sincronización, principalmente wait() y notify(), se hará sobre dicho recurso compartido por ambos hilos (podemos tener más de 2 hilos usándolo, en cuyo caso también podemos optar por notifyAll()).

En términos generales y suponiendo que el hilo actual de ejecución posee la llave sobre un determinado recurso compartido:

Si éste pasa a WAITING, **suelta la llave** del recurso compartido. El planificador puede dársela a otro hilo para que lo pueda usar.

Si se pone en SLEEPING o vuelve a RUNNABLE, **la llave la no la suelta**, imposibilitando al resto de hilos que también trabajen sobre dicho recurso compartido, el poder hacer uso de él.

No debemos olvidar que por cada elemento compartido existe una única llave, pero que un mismo hilo puede hacer uso simultáneamente de 2 o más recursos compartidos (también ninguno), es decir, estar en posesión de dos o más llaves. Así, sólo en el momento de pasar a WAITING, es cuando soltará todas y cada una de dichas llaves.

SINCRONIZACIÓN DE UN BLOQUE DE CÓDIGO CON RESPECTO A UN RECURSO COMPARTIDO (OBJETO):

Se hace necesario utilizar un objeto, el recurso compartido, a partir del cual obtener la llave (key lock) que permita o deniegue la ejecución del bloque de código sincronizado del hilo (cada hilo tiene el suyo y lo que se busca es que se pueda o no ejecutar el mismo en un momento determinado).

En el instante en el que un hilo adquiere la llave, consigue el permiso de ejecución de su bloque sincronizado, denegándose la misma al resto de hilos (cada uno de su propio bloque). El hilo elegido, puede decidir ponerse en WAITING (habrá que vigilar la notificación previa notify() – notifyAll() que debe enviar al planificador para evitar dead locks) y permitir así, que otro hilo ejecute su bloque sincronizado.

Así, la sincronización se obtiene mediante el uso de los métodos wait(), notify() y notifyAll() sobre el recurso compartido.

Esta forma de trabajo es propia de las aplicaciones productor – consumidor.

Ejemplo de un hilo productor, sobre un recurso compartido vaso:

```
public class Camarero extends Thread {

   private Vaso vaso;

   Camarero(String nombre, Vaso vaso) {
      super(nombre);
      this.vaso = vaso;
   }

   @Override
   public void run() {
      //System.out.println("SOY CAMARERO .... " + Thread.currentThread().getName());

      synchronized (vaso) {
         while (Vaso.contador < 5) {
         if ("lleno".equals(vaso.consultar())) {
            vaso.notify();
            try {
               vaso.wait();
            } catch (InterruptedException ex) {
               // nada
            }
         } else {
            vaso.llenar();
            System.out.println("EL CAMARERO LLENA EL VASO");
            vaso.notify();
            try {
               vaso.wait();
            } catch (InterruptedException ex) {
               // nada
            }
         }
         vaso.notify();
         }
      }
   }
}
```

Si un hilo no dispone de la llave para poder ejecutar su bloque de código sincronizado, no se salta dicho bloque para seguir ejecutando el resto del código de su método run(), sino que se detiene hasta poder conseguir la llave.

SINCRONIZACIÓN DE UN MÉTODO:

De la misma forma que se pueden sincronizar dos o más hilos con respecto a un recurso compartido, podemos también realizar dicha sincronización utilizando un método. Para ello, simplemente marcaremos el mismo como synchronized y pasaremos una copia de la referencia del objeto instanciado para invocarle, a todos y cada uno de los hilos que quieran usarlo.

En el momento en que un hilo acceda a dicho método, intentará adquirir la llave de sincronización y no la devolverá hasta que no termine la completa ejecución del mismo. Si obtenida la llave, dicho hilo se desactiva sin haber finalizado el método (paso a Runnable por conmutación de hilo), el resto de hilos tendrán que esperar a que éste termine, antes de poder acceder al método.

Para ello, el hilo poseedor de la llave tendrá que volver a ser elegido, entrar en el estado Running y finalizar por completo la ejecución del código del método antes de devolver la misma. Es este bloqueo el que asegura la sincronización.

En una clase podemos combinar métodos sincronizados con métodos no sincronizados.

Ejemplo:

```
synchronized void escribir() { System.out.println("entrando en
      escribir"); System.out.println("HILO : " +
      Thread.currentThread().getName());
      System.out.println("BLOQUEO EL ACCESO A LOS OTROS HILOS A ESTE MÉTODO HASTA QUE YO TERMINE");

      try {                                          // PONER A DORMIR EL HILO ACTUAL, NO SUELTA LA LLAVE.
                  Thread.sleep(4*1000);
            } catch (InterruptedException ie) {
                  ie.printStackTrace();
      }

      System.out.println("YA HE TERMINADO, LIBERO EL ACCESO (KEY LOCK - LLAVE)");
      System.out.println("saliendo de escribir");
      System.out.println("\n");
   }
```

SITUACIONES DE BLOQUEO (BLOCKED) Y BLOQUES MUERTOS (DEAD LOCKS):

Como ya sabemos, dos o más hilos pueden compartir un determinado recurso. Para evitar los problemas de sincronía o multiacceso, obligamos a los hilos a coordinarse entre ellos, a través del planificador, empleando los métodos wait(xxx), notify() y notifyAll(). Si dicha coordinación no está bien planteada, podríamos entrar en la generación de una situación de bloqueo o en un bloque muerto(es un caso particular).

Una situación de bloqueo (hilos "Blocked") podríamos entenderlo como una espera infinita. Se da la circunstancia de que **TODOS** los hilos sincronizados están en situación **WAITING** a la espera de que el planificador decida levantar uno de ellos (o todos a la vez) para poder pasarle al estado RUNNABLE y a continuación poder ser elegido para continuar con la ejecución de su método run(), en el ya conocido estado RUNNING.

El problema viene dado porque el planificador debe a su vez esperar a que sea uno de dichos hilos, el que le envié una petición, aviso o notificación de que puede proceder al levantamiento de otro hilo (puede que sea este mismo hilo que ha lanzado la petición el que resulte elegido). Dado que todos ellos están esperando a que sea otro el que genere el aviso, se concluye en que NADIE finalmente lo provocará, con lo que el planificador no podrá elegir ningún hilo para ponerlo en RUNNING, ya que no tiene ninguno para escoger (recordemos que todos estaban en WAITING).

Así, nunca se sale del estado de eterna búsqueda de un hilo disponible (RUNNABLE) por parte del planificador, lo que nos obligará a detener el programa de una forma abrupta.

Los bloques muertos pueden verse como "cruces en las llamadas de bloques sincronizados" sin que los hilos estén en waiting. Veamos cómo pueden producirse:

El hilo1 entra al método sincronizado m1() de la clase A, para lo cual adquiere su llave, comenzando a ejecutar su código. Llega un momento en el que el Scheduler conmuta el hilo con lo que se activa el hilo2, que entra al método sincronizado m2() de la clase B, adquiriendo su llave (observar como hay dos objetos, luego hay dos llaves, y que los métodos están sicronizados a nivel de objeto (this)). El hilo2 empieza a ejecutar el m2(). Un tiempo después, se vuelve a producir un cambio de hilo, con lo que hilo1 vuelve a estar activo. Éste, sigue ejecutando el método m1() y llega a una línea de código en el que debe ejecutar el método m2() del objeto de clase B, objeto compartido con el hilo2 y del que éste último tiene su llave. El Scheduler detecta esta situación, hilo1 no puede continuar porque no tiene acceso a m2() (la llave la tiene el hilo2), forzando esta situación al Scheduler a nuevamente conmutar el hilo. Así, se vuelve a activar el hilo2 y continua con su ejecución del m2() llegando a una línea en la que tiene que invocar el método m1() del objeto de clase A, que comparte con el hilo1. Aquí se repite la situación anterior, hilo2 no puede usar m1() porque la llave de acceso la tiene el hilo1, ya que fué él el que entró primero y aún no ha terminado de ejecutar el método en cuestión.

Vemos por tanto, como hilo1 espera a que hilo2 suelte la llave de m2 del objeto compartido de clase B, mientras que hilo2 espera a que hilo1 suelte la llave del m1() del objeto compartido de clase A ... ESTAMOS EN UN CALLEJÓN SIN SALIDA O DEAD BLOCK. El Scheduler no puede resolver esta situación, NO LE QUITA LA LLAVE A NINGUNO DE LOS DOS HILOS, con lo que el programa NÚNCA PUEDE CONCLUIR. Podríamos decir aquello de "yo te doy y tu me das, pero dame tú primero".

Para simular esta situación necesitamos por ejemplo, 2 clases A y B, con sus métodos m1() y m2(), respectivamente, sincronizados a nivel de objeto (se usa el propio objeto ... this implícito) y pasar la referencia de los objetos a y b creados, a ambos hilos hilo1 e hilo2. Usaremos un Thread.sleep(2000); para provocar la conmutación una vez que los hilos entran en sus respectivos métodos. Observaremos como aquello no acaba núnca.

MÉTODOS PROPIOS DE THREAD:

join() Aquel hilo del que se llama a su método join(), debe completarse para poder continuar la ejecución del hilo actual (el que hizo la llamada).
Puede lanzar InterruptedException.

join(long ms)
El hilo que llama al método join(ms) de otro hilo, esperará al menos el tiempo indicado, antes de continuar su propia ejecución.
Puede lanzar InterruptedException.

sleep(ms) El hilo que invoca al método sleep(ms) (ha de ser sobre el hilo en ejecución, no sobre otro hilo) pasará al estado SLEEPING y permanecerá así, al menos el tiempo indicado. El planificador lo volverá a colocar automáticamente en RUNNABLE.
Es estático así que podemos invocarlo sobre el hilo actual siendo lo mejor llamarlo a partir de la clase Thread.
Puede lanzar InterruptedException.

yield() Pasa el hilo a RUNNABLE, sugiriendo al planificador que ponga en ejecución otro hilo de su misma prioridad.
Es estático así que podemos invocarlo sobre el hilo actual siendo lo mejor llamarlo a partir de la clase Thread.

setName() Da un nombre o indentificador a cada objeto de la clase Thread creado. Esto también puede hacerse utilizando el constructor o mediante un super("nombre") en la clase heredera de Thread.

getName()
Devuelve un String con el nombre que se utilizó para designar el hilo. Si no se le dió nombre, obtendremos algo como ... Thread-0.

setPriority()
Establece la prioridad de ejecución de un hilo ante el planificador. En realidad se trata de un valor int que cubre el rango 1 ... 10 y que puede verse también mediante las constantes MIN_PRIORITY(1), NORM_PRIORITY(5) y MAX_PRIORITY(10).

Como ya hemos indicado, el planificador evalúa muchos parámetros a la hora de decidir cuál va a ser el hilo que efectivamente sea elegido para ejecución. El asignar una prioridad alta, SÓLO SUPONE aumentar las posibiliades de que un determinado hilo sea elegido.
Como norma a seguir, el planificador tratará siempre de ejecutar el hilo de mayor prioridad.

isAlive() Mediante un booleano, se indica si el hilo en cuestión sigue vivo o no, es decir, si ha finalizado completamente su método run().
Un hilo en estado new (sólo se ha creado el objeto de la clase que hereda de Thread) dará false al invocar sobre él a isAlive().

currentThread()
Devuelve una referencia al hilo que se está ejecutando en el momento actual.

EXCEPCIONES DE THREAD:

InterruptedException	Métodos join(), wait(xxx) y sleep(xxx)
IllegalThreadStateException	Si tratamos de volver a activar un hilo muerto o arrancar un hilo ya iniciado.

Los métodos notify() y notifyAll() no generan ninguna excepción, pero recordar que junto con wait(), SÓLO PUEDEN UTILIZARSE EN UN CONTEXTO SINCRONIZADO (el compilador no se quejará pero obtendremos una IllegalThreadStateException, lo mismo que si no sincronizamos sobre el elemento compartido).

EL MÉTODO MAIN

El propio método main (publis static void main(String args[]) {}) es en si mismo un hilo. Así, podemos crear en su interior otros hilos y lanzarlos a través de su correspondiente método star(). Igualmente, podemos hacer que main no finalice hasta que no se complete uno o más hilos, usando el join() sobre los mismos.

Cuando desde main invocamos el método run() de una determinada instancia de Thread, no estamos indicando al planificador que se encargue de gestionar dicho objeto como un hilo. En su lugar, el cuerpo o código del método run() invocado se ejecutará como parte del propio hilo main y NO como un hilo independiente.

Así, es necesario usar el método start() para que el planificador gestione la instancia del Thread como un verdadero hilo.

La ejecución del método run() de un hilo suele ser extremadamente rápida, por lo que si arrancamos distintos hilos para comprobar su funcionamiento, puede darse el caso de que el tiempo de CPU que asigne el planificador a cada de ellos, sea lo suficientemente alto como para que estos se completen, sin observarse en pantalla una mezcla de las distintas respuestas (cada una correspondiente a la ejecución de un determinado hilo). Si esto sucede, podemos optar por utilizar retardos (con sleep(xxx)) para comprobar cómo el planificador conmuta la ejecución de los mismos produciéndose dicha mezcla de respuestas.